

What we have built, and how it compares

A review of five external reference points against the Snap Apps document engine — what is running today, what is planned, and the four things this comparison changes.

DATE	STATUS	AUDIENCE	SOURCE DOCS
12 September 2026	Review draft	Engineering & non-technical	docs/OCR.md, docs/AI.md, bench/results

The short version. Five things were examined: a benchmark comparing three ways of serving the same small AI model; a teaching demo showing how compressing a model changes its speed and accuracy; a scan of the main public library of open AI models; **PaddleOCR**, the toolkit our own reading stage is built on; and **ai-parsing**, a live sibling system solving almost exactly our problem the opposite way.

None is a commercial competitor. All five are useful, because each does something we have written down as important and not yet finished.

Two findings change a decision. **We are one model generation behind inside a library we have already installed** — the upgrade is a one-line change and claims roughly 5% better reading. And **the two open models our architecture document names as strongest are licensed so we cannot ship them to a customer.**

01 Why this review exists

Snap Apps reads Australian receipts and invoices and turns them into accounting records. The part that does the reading is called the **document engine**. Its architecture is written down in `docs/OCR.md`, a long and deliberately honest document whose competitive section opens with a table of how well rival systems perform — and then admits, in bold, that *every number in that table is someone else's*.

Five links were put forward for comparison. This document answers the question properly: what are they, what do we have, what is planned, and where does the comparison actually bite.

IN PLAIN TERMS

Reading a receipt with a computer sounds simple and is not. The difficulty is not reading the letters — it is knowing, afterwards, whether you read them correctly. A system that is confidently wrong about a tax figure is worse than one that says "I could not read this." Almost everything below is about that distinction.

The three references at a glance

REFERENCE	WHAT IT IS	WHAT IT ACTUALLY DOES	RELATIONSHIP TO US
sentiment.net "Experiment 001"	A public benchmark, not a product	Takes one sentence and one fixed AI model, then runs it through three different software stacks on identical hardware to see which serves it fastest.	Not a competitor. A method worth copying.
github.com/azlanabas/utmcslmv	A university teaching demo	Runs one small language model at three compression levels side by side so students can see the trade-off between model size, speed and accuracy.	Not a competitor. Fills a measurement gap we have.
Hugging Face search "OCR for documents, Australia"	The main public library of open AI models	Shows what the world has already published and made free to use.	Tells us what to build and what to borrow .
PaddlePaddle/PaddleOCR 89,400 stars	The open toolkit our reading stage is built on	Text detection and recognition, Apache-2.0, used by a large part of the industry. We pin it in our own dependency file.	Our supplier — and, at its top end, a competitor.
aiparsing.gaiada.online "ai-parsing"	A live sibling system	Invoice extraction in production, solving nearly our problem with the opposite architecture — an AI model makes the routing decisions rather than deterministic code.	The closest thing we have to a control group .

NOTE ON CATEGORY

The first two are small personal or academic projects — one has two commits and no stars. Nothing below treats them as rivals. They are treated as evidence, because a small project that has actually measured something outranks a large document that has not.

The last two are different in kind and the most useful of the five. One is code we already ship. The other is a system built by the same people, to the same brief, on the opposite principle — which makes it the nearest thing to a controlled experiment this review contains.

02 Where the Snap Apps engine stands today

Before comparing anything, an honest statement of our own position. The engine has three layers, and they are at three very different stages of completion.

CAPABILITY	STATUS	WHAT EXISTS IN THE CODEBASE	WHAT THAT MEANS IN PRACTICE
Reading the document	LIVE	A vision-language model is handed the whole page and asked for structured data. Default gemma4:31b; a router escalates through four models on failure.	Works. But the model never reports <i>where</i> on the page it found a number — it goes straight from picture to answer.
Checking the reading	LIVE	Arithmetic, date and ABN checks in <code>validators.ts</code> , including the modulus-89 ABN checksum and a dual-convention line-total check.	This is our strongest asset and it is fully built. Deterministic code, not a model, decides whether a human must look.
Audit trail & replay	LIVE	Migration 0009: re-extraction updates in place, both runs survive, human corrections are locked, documents behind a posted transaction are frozen.	A past result can be reproduced and every correction is traceable. Rare in this field.
Data isolation	LIVE	Row-level security in the database; the background worker itself goes through it rather than around it.	One tenant cannot read another's documents, enforced by the database, not by application discipline.
Our own OCR stage	SHADOW	<code>packages/docai</code> plus a Python service wrapping PP-OCRv5. Built, tested, and wired in — but only in shadow mode.	It runs <i>alongside</i> the real result and never affects it. If the switch is unset, it is not called at all.
Location of every value	SHADOW	<code>grounding.ts</code> attaches coordinates to extracted fields — advisory only.	Cannot yet be relied on, because the stage that produces coordinates is not the primary reader.
Deployment restrictions	LIVE	<code>registry.ts</code> models residency as a <i>type</i> . An air-gapped profile does not forbid external engines — it removes them from the list.	A misconfigured flag cannot send Australian tax records offshore, because the option does not exist in that build.
Calibrated confidence	GAP	The engine self-reports <code>calibrated: false</code> . The cutoff of 0.8 is labelled in code as a placeholder.	When the system says it is 95% sure, that number does not yet mean anything measurable.
Evaluation corpus	GAP	Four documents, all synthetic, rendered from HTML templates in the bench folder. The manifest says so explicitly.	Good enough to exercise the harness. Not good enough to claim accuracy on real Australian paperwork.

Status as of 12 September 2026. "Shadow" means the code runs but its output is stored separately and never changes what the user sees.

The single most important line in this table: **the OCR stage is built but not switched on**. Every stored layout record from the most recent run reads `ppocr-v5 failed: docai-engine sidecar not reachable`. Shadow mode that never executes collects no evidence, and the calibration work planned next depends entirely on that evidence.

What we have actually measured

As of 11 September 2026 the competitive table is no longer entirely borrowed. Two benchmark runs exist, on our own harness, on our own documents.

4 × 4 × 5	0.875	0.0000	n/a		
documents × models × repeated runs in the extraction benchmark	text-detection recall of the OCR stage — 7 of 8 regions found	character error rate, ignoring spacing — the characters are read perfectly	confidence-vs-error correlation — not enough data to compute		
MODEL	EASY RECEIPT	HARD RECEIPT	NO-ABN RECEIPT	2-PAGE INVOICE	FIELD OUTCOMES
minimax-m3	4.2 s	5.7 s	3.4 s	13.0 s	Clean — no wrong, no misses, both abstentions correct
kimi-k3	5.9 s	7.7 s	5.4 s	28.6 s	Clean — same accuracy, roughly twice the time
gemma4:31b (default)	2.4 s	2.9 s	1.8 s	4.1 s	Fastest — but 2 wrong and 1 miss across the set
qwen3.5:397b	21.2 s	27.3 s	20.4 s	50.2 s	5–10× slower for no accuracy gain over minimax-m3

Median seconds over 5 repeated runs, 11 September 2026. Documents are synthetic. "Abstention correct" means the document genuinely had no ABN and the model correctly returned nothing rather than inventing one.

WHAT THIS TABLE IS WORTH

It is real, it is ours, and it is small. Four synthetic documents cannot tell us how the system behaves on a crumpled thermal docket photographed in a car park. That corpus is the next thing to build, and the architecture document already says so.

It does, however, settle one live question: the largest model is not the best one. **qwen3.5:397b costs five to ten times the latency of minimax-m3 and reads no better.**

What is planned

PHASE	EFFORT	STATE	WHAT IT DELIVERS
0 — Stop lying to the user	1–2 weeks	DONE	Multi-page capture fixed, PDF intake accepted, evaluation harness built. Gate passed 11 Sep 2026 — and found six defects no unit test had caught.
1 — Reading stage	4–6 weeks	NEXT	Promote the OCR stage from shadow to primary. The AI model is demoted from <i>the reader</i> to <i>the fallback for hard patches</i> . Calibrated confidence.
1b — Device tier	2–3 weeks	PLANNED	The same reading stage running on a phone, offline.
2 — Tables	4–6 weeks	PLANNED	Business documents with real table structure.
3 — Figures	3–4 weeks	PLANNED	Charts and their underlying series.
4 — Forms	4–6 weeks	PLANNED	Filled-in forms and handwriting.
5 — Portability	ongoing	PLANNED	On-premises packaging, and the corrections flywheel: every human fix becomes training data.

The four-tier routing model

Both planned and partly live. The principle is **cheapest capable engine first, escalate only on failure** — implemented today in `pipeline.ts`.

TIER	WHAT READS THE PAGE	RUNS WHERE	COST PER PAGE	STATE
T0	Text already embedded in a PDF	Anywhere	Free, and exact	LIVE
T1	Our own OCR stage (PP-OCrv5)	Ordinary CPU, offline, phone	Near zero	SHADOW
T2	Open AI model on our own GPU	Our cloud, customer's cloud, air-gapped	GPU-seconds	PLANNED
T3	Frontier AI model (Claude, Sydney region)	Cloud deployments only	~\$6–11 per 1,000 pages	LIVE

Today almost everything goes to T3 because T1 is not switched on. The economics of the whole plan depend on reversing that: T1 handling most of the page, T3 seeing only the genuinely hard 5–15%.

03 Reference one — the sentiment benchmark

sentiment.net1io.space is an experiment, presented as one. It has sections headed *Apparatus*, *Method*, *Observation* and *Control*. One sentence goes in; one fixed model classifies it as positive or negative; the only thing that changes is the software stack serving it.

HELD CONSTANT	DETAIL
Model	distilbert-base-uncased-finetuned-sst-2-english — a small, standard sentiment classifier
Hardware	One rented server, 2 CPU cores
Interface	POST /analyze — identical for all three stacks

VARIABLE	STACK	WHAT IS BEING TESTED
Stack A	FastAPI + Python (PyTorch)	The conventional approach — and the one our own OCR service uses
Stack B	Rust / Axum bridging to Python	A fast front end still calling Python for inference
Stack C	Rust / Axum + rust-bert (native)	No Python at all in the serving path

Comparison

DIMENSION	THEIR EXPERIMENT	SNAP APPS — CURRENT	SNAP APPS — PLANNED
Question asked	Which serving stack is fastest?	Which AI model reads a receipt best?	Unchanged, plus: is our own OCR stage good enough to take over?
Scoring	Latency	Per-field, with <i>correct abstention</i> as a distinct success and <i>hallucination</i> as worse than a miss	Adds calibration: does "95% sure" mean right 95% of the time?
Repeats	Live, per request	5 repeats, medians reported	Same, on real documents
Missing-engine handling	n/a	Reports "not run" with a reason; never invents a score	Unchanged
Serving layer tested	Yes — it is the whole point	Never	Not in the plan

Our harness is the more sophisticated instrument. Their *discipline* is the thing worth taking: **hold everything constant, change exactly one thing, publish the number you measured**. On the one dimension they chose, they have data and we have none.

Why the serving layer is not an idle question for us

Our OCR service is Python and FastAPI — their Stack A. It currently runs with a CPU accelerator switched off, because PaddlePaddle 3.3.1 throws an error on both detection models with it enabled. The workaround is documented and correct. It also means the stage is running slower than its ceiling, and we have never measured what a different serving architecture would recover.

IN PLAIN TERMS

Two parts of the system could be responsible for slowness: the AI itself, or the plumbing around it. We have measured the AI. We have never measured the plumbing. This experiment is a template for doing that in about a day.

04 Reference two — the quantization demo

`utmcs_llmv` is a university teaching tool, deployed 6 September 2026, with two commits and no stars. It demonstrates **quantization**: compressing an AI model so it is smaller and faster, at some cost to accuracy. It runs one model — Llama 3.2 1B — at three compression levels and lets a student watch the trade-off.

IN PLAIN TERMS

Quantization is like saving a photo as a smaller JPEG. The file shrinks and loads faster; at some point the picture starts to look wrong. The skill is knowing where that point is. For AI models it matters enormously, because the difference between compression levels can be the difference between needing an expensive GPU and not.

Their architecture, and ours

CONCERN	UTMCS_LLMV	SNAP APPS
Front end	Next.js, TypeScript	Next.js web, React Native mobile
Back end	Rust / Axum	Node / Fastify, plus a Python OCR service
Store	SQLite, append-only	PostgreSQL with row-level security, append-only edit history
Where AI runs	A Mac on the desk, reached over a private network	Hosted models today; our own CPU stage in shadow
Separation principle	The web server never runs a model — it stores results and proxies when the Mac is awake	The same idea, expressed as a <i>type</i> : a deployment profile filters the engine list, so a forbidden engine is absent rather than disabled

WHERE WE ARE AHEAD

Ours is the stronger design. Their separation is a network arrangement that can be misconfigured; ours is enforced in code — `permitted()` removes an engine from the list entirely. Their store has no audit trail, no locked human corrections, no tenant isolation. Ours has all three, tested against a real database.

WHERE THEY ARE AHEAD

Theirs is **running**. Ours is behind an unset environment variable, and every stored record says the service was unreachable. A split-brain design that has never completed a run has not been proven — it has been described.

The gap this exposes

Our tier table names "an open AI model on our own GPU" as tier 2 and never pins which model, at which compression. GPU cost is the largest number in the plan's cost section. Meanwhile, in our engine registry, the field recording how

long each engine takes reads:

```
medianSeconds: number | null; /* Measured median seconds per page. null until someone measures it. */
```

QUESTION	THEM	US — CURRENT	US — PLANNED
Effect of compression on speed	Measured, 3 levels	Never measured	Not in the plan
Effect of compression on accuracy	Measured	Never measured	Not in the plan
Cost per page at tier 2	n/a	Estimated, not measured	Still an estimate
Per-engine timing in the registry	n/a	null	To be filled by the harness

A two-commit teaching repository has plotted the curve that our cost model assumes and has never drawn. The harness to draw it already exists in our codebase. This is the cheapest high-value measurement available to us.

05 Reference three — the public model library

Hugging Face is where the world publishes open AI models. The search page is built in a way that hides results from automated retrieval, so the site's data interface was queried directly instead. Two findings, and the second changes a decision.

Finding one — nobody has built this for Australia

SEARCH	BEST RESULT	WHAT IT ACTUALLY IS
"australian" — models	isaacus/open-australian-legal-gpt2	Legal text generation. Also returned: text-to-speech voices, and a dog-breed image model.
"australian" — datasets	ATO-Australian-Tax-Rulings	Tax <i>rulings as text</i> . Useful, but not documents to be read.
"receipt" — models	SZL-Forge-1.5B-ReceiptAgent	1,970 downloads. The rest of the top ten are largely Japanese receipt models, one German.

No model anywhere in the public library is trained to read Australian receipts, invoices or tax documents.

Meanwhile the general "read any page" category is saturated:

MODEL	DOWNLOADS	LICENCE	CAN WE SHIP IT TO A CUSTOMER?
datalab-to/chandra-ocr-2	2,908,785	OpenRAIL	No — fails our licence floor
baidu/Unlimited-OCR	2,700,194	MIT	Yes
deepseek-ai/DeepSeek-OCR	2,400,151	MIT	Yes
zai-org/GLM-OCR	1,893,869	MIT	Yes
datalab-to/surya-ocr-2	1,206,466	OpenRAIL	No — fails our licence floor
PaddlePaddle/PP-OCRv5	763,379	Apache-2.0	Yes — this is what we already use
stepfun-ai/GOT-OCR2_0	670,426	Apache-2.0	Yes
dots-studio/dots.ocr	368,761	MIT	Yes
nanonets/Nanonets-OCR-s	351,066	none stated	No — unstated licence is an unusable licence

Licences verified against the Hugging Face model index, 12 September 2026. Our rule allows only Apache-2.0 or MIT weights in a build handed to a customer.

Finding two — the two strongest models are ones we cannot ship

Our architecture document singles out two open systems by name: **Chandra 2** as the "best single published number", and **Surya 2** as the "best cost/accuracy of the small models". Both are OpenRAIL-licensed. Our own licence rule — written into code as a function that scans the engine list and returns violations — would reject both from any package sent to a customer.

IN PLAIN TERMS

A licence is the legal permission attached to a model. Some open licences let you use a model freely but not redistribute it inside a product you sell. OpenRAIL is that kind. For a consumer app that calls a model over the internet it rarely matters. For an enterprise customer who wants the system installed inside their own building, it decides whether the product can exist at all.

Our own document anticipated this in general — it warns that discovering a licence problem during a customer's procurement review is "a bad day that CI can have instead." It just never named which two models the warning applies to.

The comparison table in `docs/OCR.md` §2 currently reads as a menu of options. Two of its headline entries are not on the menu. A licence column should be added, so nobody plans around a model we cannot deliver.

What the saturation tells us

STRATEGIC QUESTION	WHAT THE LIBRARY SHOWS	CONSEQUENCE FOR US
Should we compete on general page reading?	Four models with over 1.8 million downloads each, heavily funded, measured to two decimals	No. We would start a year behind. Our plan already says so; the download counts confirm it.
Is there an Australian model to borrow?	Nothing. Not one.	The shelf is empty because only someone with the data can fill it.
Do we have that data?	n/a	Yes. Every human correction in the review screen has been stored since migration 0004 as a matched triple: the image, what the machine read, what the person changed it to.
Is the tier-2 slot fillable under our licence rule?	DeepSeek-OCR, GLM-OCR and dots.ocr are all MIT and all widely used	Yes. Losing Chandra and Surya is a narrowing, not a blockage.

06 Reference four — PaddleOCR, our own supplier

PaddleOCR is the most widely used open text-recognition toolkit in the world: 89,400 stars, Apache-2.0, supporting over 100 languages. It is not a competitor in the ordinary sense. **We ship it.** Our dependency file pins `paddleocr==3.7.0`, and our OCR service is a wrapper around it.

Checking it produced the most immediately actionable finding in this review.

We are current on the library and one generation behind on the models

ITEM	WHAT WE USE	WHAT IS AVAILABLE	VERDICT
Library version	<code>paddleocr 3.7.0</code>	3.7.0	Current — this is the newest release, 11 June 2026
Detection model	<code>PP-OCRv5_server_det</code>	PP-OCRv6 (3 tiers)	Superseded
Recognition model	<code>PP-OCRv5_server_rec</code>	PP-OCRv6 (3 tiers)	Superseded
Engine identifier in code	<code>ENGINE_ID = "ppocr-v5"</code>	—	Hardcoded to the older generation

PP-OCRv6 is the headline feature of release 3.7.0 — the exact version already pinned in `services/docai-engine/requirements.txt`.

IN PLAIN TERMS

We already installed the toolbox that contains the newer, better tools. Our code just asks for the older ones by name, because it was written before the new ones arrived. Changing which tools it reaches for is a one-line edit, and the licence and cost do not change at all.

What the newer generation offers

TIER	SIZE	INTENDED FOR	CLAIMED BY PADDLEOCR
tiny	1.5M parameters	Edge devices, browser	6.1× faster on an Apple M4 chip
small	7.7M parameters	Mobile	—
medium	34.5M parameters	Server — our tier	+4.6% detection, +5.1% recognition over the model we currently run; 5.2× CPU speedup via OpenVINO; 0.13s per page on an A100 GPU

Figures are PaddleOCR's own README claims, not measured by us. Also claimed: a single model now covering 50 languages, and — at 34.5M parameters — outperforming far larger general-purpose vision models on document text.

Three consequences for us

	CONSEQUENCE	DETAIL
1	Our weakest measured number is the one the upgrade targets	Our OCR stage found 7 of 8 text regions — a detection recall of 0.875. Detection is precisely what the new generation claims to improve by 4.6%. This is testable against the benchmark we already have.
2	There may be a way around our disabled accelerator	Our service runs with its CPU accelerator switched off, because of a crash in one specific acceleration library. PaddleOCR's 5.2× speed claim is quoted for OpenVINO — a different accelerator entirely , which we have never tried. The workaround we documented may have an alternative we never looked for.
3	The phone tier now has an obvious candidate	Phase 1b plans the reading stage running offline on a phone and names no model. A 1.5M-parameter tier with a published Apple-silicon speed figure is exactly that shape. Our code currently offers only two options, <i>mobile</i> and <i>server</i> , both pointing at the older generation.

The uncomfortable half: PaddleOCR is also a competitor

The same project publishes **PaddleOCR-VL**, a document-parsing model claiming 96.3% on a major public benchmark — and it is Apache-2.0, so unlike Chandra and Surya we *could* ship it. That model competes with our entire reading stage, not just part of it.

This is not a threat, provided we are honest about where we differ. Our architecture document already says it plainly: **do not compete on turning pages into text — use the people who are winning that race.** Our differentiation is knowing when we are wrong, proving where every number came from, and being deployable where the internet is forbidden. None of those are things PaddleOCR-VL offers, and none of them are reading accuracy.

07 Reference five — ai-parsing, the control group

aiparsing.gaiada.online is live, and titled "ai-parsing — Invoice Processing Dashboard". It is a sibling system: same problem, same brief, built on the opposite principle. Our architecture document already analyses it at length as "the nearest prior art, and it is instructive."

HOW THIS WAS CHECKED

The live site is a single-page application, so an automated fetch returns only its title. The analysis below comes from our own architecture document, which examined the system **at source** — reading its code, not benchmarking its output. An attempt to independently re-verify one finding against the live deployment was blocked by this environment's safety controls, and was not worked around. Anything marked below as a source reading should be re-confirmed directly before being acted on.

Same problem, opposite architecture

DECISION	AI-PARSING	SNAP APPS
Reading the page	LlamaParse, agentic tier — a paid external service	Vision model today; our own OCR stage next, self-hosted and free per page
Understanding it	deepseek-chat across four of five reasoning steps	A model reads; deterministic code decides
Who decides if a human looks	The language model. The two thresholds are real constants — flag at 75, escalate at 50 — but they are inserted as text into the model's own instructions. No comparison is performed in code.	Code. Arithmetic, date and ABN checksum rules in <code>validators.ts</code> . The model has no vote.
Confidence	The model scores its own homework	Uncalibrated today and treated as advisory; calibration is planned and measurable
Access control	Reported at source as fully open on the invoice collection — all records readable without signing in	Row-level security in the database; the background worker goes through it, not around it
Model choice	Built on the one model our own testing excluded	Excluded in code — our benchmark found it called the right tool and then ignored the answer it got back

What its own numbers show

Five hand-built test documents, thirty-four runs. The confidence column is the model rating itself — not measured accuracy.

TEST DOCUMENT	SELF-REPORTED CONFIDENCE	OUTCOME
clean_typed_invoice	95–97	Auto-approved
scanned_crumpled_receipt	62–68	Flagged for review
multipage_buried_table	95	Flagged for review
unrecognized_vendor	95	Escalated
handwritten_correction	40–45	Escalated

Consistent across repeats — but that is the consistency of a demo, not evidence of accuracy. The prompts were written against these same five fixtures.

The one genuinely measured quantity in that system is the deterministic one. Its arithmetic cross-check caught a line total of **\$518.00 against a stated \$512.00**, and **\$760.00 typed against \$700.00 struck through** by hand. Cheap arithmetic found what a self-reported 95% confidence did not.

WHY THIS MATTERS MORE THAN THE OTHER FOUR REFERENCES

This is a live system, built to the same brief, that demonstrates the exact failure our architecture is designed to prevent — and demonstrates it while reporting high confidence. It is the strongest evidence in this review for our central design choice, and it did not come from a benchmark or a vendor claim. It came from reading a sibling system's source.

It also shows the cost of the opposite choice honestly: the parts of ai-parsing that worked are the parts that were arithmetic, and the part that failed is the part that asked a model to police itself.

The warning we should take personally

Two of the four problems recorded against ai-parsing are not really about reading documents at all. One is that a decision written down as a rule was never implemented as one. The other is that access control was written permissively and never tightened.

Our own repository closed a defect of precisely that shape on the day this review was written. The commit message begins: *"Two decisions that were written down, believed, and never applied."* A sign-in endpoint that was meant never to reach production had its production guard applied to the wrong route; and a model exclusion recorded in our AI document had never been enforced in code.

The pattern is the finding. This engine's strength is that its important decisions are written down with reasons. Its recurring weakness — visible in ai-parsing, in the unenforced exclusion, in the sign-in guard, and in the OCR stage sitting behind an unset switch — is that **writing a decision down is being mistaken for shipping it.**

08 The comparison in one table

The dimensions that matter to a finance customer, across all six systems. Three of them are research demos or toolkits and are not expected to score on most rows — they are included to show what each kind of system does and does not have to care about. The column that matters most is **ai-parsing**: it is the only other one built to do our actual job.

CAPABILITY	SENTIMENT EXPERIMENT	UTMCS_LLMV	OPEN OCR MODELS	PADDLEOCR (OUR SUPPLIER)	AI-PARSING (CONTROL GROUP)	SNAP APPS TODAY	SNAP APPS PLANNED
Reads Australian tax documents	No	No	No	No	Invoices, not AU-specific	Yes	Yes, wider
Says where on the page a value came from	n/a	n/a	Block level	Yes — that is its job	No	Shadow only	On every value
Confidence that means something	No	No	Not reported	Per-character, raw	Model rates itself	Not calibrated	Calibrated
Can say "I could not read this"	No	No	Largely absent	Yes	Fills the gap	Scored as success	Yes
Who decides a human must look	n/a	n/a	n/a	n/a	The language model	Code	Code
Catches its own mistakes	No	No	No	No	Arithmetic only — and it worked	Arithmetic + ABN checksum	Plus engine disagreement
Reproduce a past result exactly	No	Results stored	No	Deterministic	No	Versioned runs	Yes
Audit trail of human corrections	No	No	No	No	No	Yes, with locking	Feeds training
Can run with no internet	Locally	Private network	Most can	Yes	Two paid APIs	Built, not on	Enforced by type
Data residency guaranteed	No concept	No concept	No concept	No concept	No concept	Engine registry	Same
Tenant data isolation	No	No	n/a	n/a	Reported fully open	Database-enforced	Same
Cost per page at scale	n/a	n/a	Near zero	Near zero	Two metered APIs	Frontier-priced	Mostly near zero
Serving stack measured	Yes — the point	Partly	n/a	Published	No	Never	Not in the plan
Model compression measured	No	Yes, 3 levels	n/a	Yes, 3 tiers	No	Never	Not in the plan
Accuracy on real captured documents	n/a	n/a	Public benchmarks	Public benchmarks	5 hand-built fixtures	Synthetic only	Real gold sets

"n/a" means the dimension does not apply to that kind of system. Red entries against the research demos are not criticism — a sentiment benchmark has no reason to have an audit trail. Entries for ai-parsing derive from a source reading recorded in our architecture document, not from testing the live system.

Two readings of this table. **Down the ai-parsing column:** on every row that decides whether a finance team can trust the output, the system built the opposite way scores badly — and its one green entry, arithmetic, is the same mechanism we rely on. **Across the last three rows:** every gap we have that these references do not is a measurement gap, not a capability gap. We have built more than any of them and measured less of it.

09 What this changes

Six actions, in the order they should be taken. Four are hours of work; two are days.

	ACTION	EFFORT	FROM	WHY
1	Start the OCR service and complete one real shadow run	Hours	Our own state	Every stored layout says the service was unreachable. Nothing downstream — calibration, grounding, the Phase 1 promotion — can proceed without evidence from a run that actually happened.
2	Point the sidecar at the newer model generation	Hours	PaddleOCR	PP-OCRv6 ships inside the version we already pin. Same licence, same cost, claimed +4.6% detection — which is our weakest measured number. A one-line change, then re-run the benchmark that already exists.
3	Add a licence column to the competitive table	Hours	Model library	The two models named as strongest cannot be shipped to a customer. Recording that now prevents a plan being built on them.
4	Fix the character-error-rate reporting	Hours	Our own state	The OCR stage scores 6.6% error strictly and 0.0% ignoring spacing — it reads \$1,042.60 as \$ 1 , 042.60. That is a spacing convention, not a misreading, and the headline number overstates the error.
5	Try the alternative CPU accelerator	Days	PaddleOCR	Our accelerator is off because of a crash in one library. PaddleOCR's published 5.2× speed figure is for a different one we have never tried. This may make the documented workaround unnecessary.
6	Run the two missing measurements	Days	Demos 1 & 2	A compression sweep for the tier-2 model, and one serving-layer comparison for the OCR service. Both are gaps the plan does not close; both have an existing harness.

Actions 1, 2 and 4 together would replace most of the uncertain numbers in this document with measured ones, in under a day.

The honest summary

The engine's design is well ahead of the research demos and, on the dimensions an accounting customer actually buys, well ahead of the open models too: provenance, calibrated abstention, deterministic validation, replay, audit trail, residency as a type. Against ai-parsing — the only other system here built to do our job — the gap is not close, and it is a gap in the things that decide whether a finance team can trust the output at all. None of that is aspiration; most of it is built and tested.

What the five references expose is narrower and fairly uncomfortable, and it is the same thing three times over:

WRITTEN DOWN

ACTUALLY IN FORCE

The OCR stage is the future primary reader	Behind an unset switch; every stored run says "unreachable"
A model our testing rejected must not be used	Enforced in code only as of this week
A sign-in bypass must never reach production	The guard was on the wrong route until this week
The strongest open models are our candidates	Two of them are unshippable under our own licence rule
We use the best available reading models	One generation behind, inside a library already installed

The fix is not more architecture. The architecture is genuinely good, and the decisions in it are right — including the one ai-parsing proves by getting wrong. The fix is to close the gap between **a decision recorded with its reasons** and **a decision running in production**: switch on the stage that is already written, ask for the models we already downloaded, run it against real documents, and fill in the numbers our own code still marks `null`.

Sources. Snap Apps repository as of 12 September 2026: `docs/OCR.md` (§2.1 for the ai-parsing source reading), `packages/docai/src/registry.ts`, `packages/docai/src/pipeline.ts`, `apps/server/src/extraction/shadow.ts`, `apps/server/bench/results/`, `services/docai-engine/`, commit `d588ff3`. External, queried 12 September 2026: `sentiment.net1io.space`; `github.com/azlanabas/utmcs_llmv`; Hugging Face model index; `github.com/PaddlePaddle/PaddleOCR` release notes for v3.7.0; `aiparsing.gaiada.online`.

Caveats. Benchmark figures are on four synthetic documents and do not represent accuracy on real captured paperwork. Download counts, licences and version numbers are point-in-time. PP-OCRv6 performance figures are PaddleOCR's own published claims and have not been verified by us. None of the five external projects was benchmarked — no number in this document was produced by running their code. Findings about ai-parsing come from a source reading recorded in our own architecture document; an attempt to re-verify one of them against the live deployment was blocked by this environment's safety controls and was not circumvented.